

Python Developer Assignment Documentation

Project Overview

This project involves **web scraping**, **storing data in MongoDB**, **building a Django admin panel**, and **analyzing salaries using NumPy**.

Below is a step-by-step guide to completing the assignment.

1. Web Scraping - Indeed Job Listings

1.1 Install Required Libraries

Use the following command to install necessary dependencies:

```
pip install requests beautifulsoup4 pymongo numpy django
```

1.2 Scraping Jobs from Indeed

Python script to scrape job listings for **"Python Developer"** from **Indeed**:

```
import requests
from bs4 import BeautifulSoup

def scrape_jobs():
    url = "https://www.indeed.com/jobs?q=Python+Developer"
    headers = {'User-Agent': 'Mozilla/5.0'}
    response = requests.get(url, headers=headers)

    if response.status_code != 200:
        print("Failed to fetch jobs")
        return []

    soup = BeautifulSoup(response.text, "html.parser")
    job_list = []

    for job in soup.find_all('div', class_='job_seen_beacon'):
        title = job.find('h2').text.strip() if job.find('h2') else "N/A"
        company = job.find('span', class_='companyName').text.strip() if job.find('span',
class_='companyName') else "N/A"
        location = job.find('div', class_='companyLocation').text.strip() if job.find('div',
class_='companyLocation') else "N/A"
```

```
salary = job.find('span', class_='salary-snippet')
salary = salary.text.strip() if salary else "N/A"
```

```
job_list.append({"title": title, "company": company, "location": location, "salary": salary})
```

```
return job_list
```

2. Store Scraped Data in MongoDB

2.1 Setting Up MongoDB (Cloud Database)

Create a **MongoDB Atlas** account and get your connection string:

```
mongodb+srv://<username>:<password>@cluster.mongodb.net/?retryWrites=true&w=majority
```

2.2 Storing Data in MongoDB

```
from pymongo import MongoClient
```

```
def store_jobs(jobs):
    client = MongoClient("mongodb+srv://<username>:<password>@cluster.mongodb.net/")
    db = client["job_listings_db"]
    collection = db["python_jobs"]

    if jobs:
        collection.insert_many(jobs)
        print(f"Inserted {len(jobs)} job listings into the database.")
    else:
        print("No jobs to insert.")
```

3. Django Admin Panel - Job Management

3.1 Create a Django Project & App

```
django-admin startproject job_project
cd job_project
django-admin startapp job_manager
```

3.2 Configure **settings.py**

Add **job_manager** to **INSTALLED_APPS**:

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'job_manager',  
]
```

3.3 Define the Job Model (**models.py**)

```
from django.db import models
```

```
class Job(models.Model):  
    title = models.CharField(max_length=255)  
    company = models.CharField(max_length=255)  
    location = models.CharField(max_length=255)  
    salary = models.CharField(max_length=255, blank=True, null=True)  
  
    def __str__(self):  
        return self.title
```

3.4 Register Model in Django Admin (**admin.py**)

```
from django.contrib import admin  
from .models import Job  
  
class JobAdmin(admin.ModelAdmin):  
    list_display = ('title', 'company', 'location', 'salary')  
    search_fields = ('title', 'company', 'location')  
  
admin.site.register(Job, JobAdmin)
```

3.5 Run Migrations & Start Server

```
python manage.py makemigrations job_manager  
python manage.py migrate  
python manage.py createsuperuser # Create an admin user  
python manage.py runserver
```

4. Calculate Average Salary Using NumPy

```
import numpy as np
```

```
def calculate_avg_salary():
```

```
    client = MongoClient("mongodb+srv://<username>:<password>@cluster.mongodb.net/")
```

```
    db = client["job_listings_db"]
```

```
    collection = db["python_jobs"]
```

```
    salaries = []
```

```
    for job in collection.find():
```

```
        try:
```

```
            salary = float(job["salary"].replace("$", "").replace(",", ""))
```

```
            salaries.append(salary)
```

```
        except:
```

```
            continue
```

```
    if salaries:
```

```
        return np.mean(salaries)
```

```
    return "No valid salary data found."
```
